

FULL-STACK TECHNICAL DOCUMENTATION

CodeLens AI

Next-Generation Interactive Code Execution Visualizer, AST
Semantic Explainer & Multilingual Debugger

Author & Developer

Dinesh .P

Version & Status

v1.0.0 (Production-Ready)

Repository

github.com/dinesh2505034/codelens-ai

Published Date

August 2026

Table of Contents

1. Executive Summary & System Overview
2. High-Level Architecture & Component Design
3. Core Technical Capabilities
 - 3.1 Native Python Trace Engine (`sys.settrace` Post-Execution Sync)
 - 3.2 In-Terminal Interactive Input Pipeline (`stdin`)
 - 3.3 7-Point Dynamic Error Diagnostic Engine
 - 3.4 Deep AST Semantic Code Explainer & Complexity Derivations
 - 3.5 Optical Character Recognition (OCR) Image-to-Code Pipeline
4. Frontend User Interface & Visualizer System
5. API Endpoints & Communication Protocols
6. Installation, Local Setup & Docker Deployment
7. Security, Sandboxing & Performance Optimization

1. Executive Summary & System Overview

CodeLens AI is a full-stack educational and professional developer tool designed to transform abstract source code into an intuitive, line-by-line interactive visualizer. Unlike standard static linters or simple code sandboxes, CodeLens AI executes actual programs in real time, monitors variable allocations in memory, dynamically intercepts interactive standard inputs (`stdin`), and produces compiler-accurate terminal streams alongside plain-English pedagogical explanations.

Real Native Execution

True Python runtime execution with live AST state inspection, eliminating hardcoded mocks and precision truncation for arbitrary-length integers.

Interactive In-Terminal Input

Seamless execution pause and resume for `input()`, `cin >>`, `scanf()`, and `Scanner` with inline terminal submission.

7-Point Dynamic Error Diagnostics

Context-aware error explanations that dissect what happened, why it failed, and generate actionable corrections with examples.

Multimodal OCR Code Scanner

Extract clean, formatted code from photos and screenshots using client/server optical character recognition with indentation repair.

2. High-Level Architecture & Component Design

The system follows a modular client-server architecture built on modern JavaScript, Python, Node.js, Vite, and TailwindCSS.



3. Core Technical Capabilities

3.1 Native Python Trace Engine (`sys.settrace` Synchronization)

Standard trace tools often suffer from a 1-step latency because Python triggers `sys.settrace` line events *before* executing the line. CodeLens AI solves this through a **Pending Frame Buffer** architecture:

- When a line event fires, the previous pending step is finalized using the updated `frame.f_locals` and standard output buffer.
- When the final line or function return executes, the return event captures the completed state, guaranteeing that Step 1 (e.g. `i = 0`) immediately shows `i: 0` in the variable memory table without lag.
- Integers exceeding JavaScript's IEEE 754 float precision ($> 2^{53} - 1$) are safely stringified so computations such as `122!` display all 203 digits without scientific notation.

3.2 In-Terminal Interactive Input Pipeline (`stdin`)

When code requires interactive user inputs (e.g. `i = int(input("hi:"))`):

1. The custom input interceptor in `pythonTracer.py` prints the prompt (`hi:`) into the output stream.
2. If input data is missing, it raises an internal `__WAITING_FOR_USER_INPUT__` signal and transmits `isWaitingForInput: true` with the prompt to the client.
3. The frontend Output Terminal highlights with an active border, shows the inline prompt, and displays an input field with a **[Submit]** button.
4. Upon submission, input values are buffered, execution traces smoothly to the next step, and variable states update in real time.

3.3 7-Point Dynamic Error Diagnostic Engine

When a runtime error occurs (such as entering `'d'` for `int(input())`), the engine intercepts the exception and constructs an exhaustive 7-point pedagogical diagnosis:

Diagnostic Point	Contextual Analysis
1. Expected Input	An integer value for variable <code>l</code> (whole numbers <code>0-9</code> , e.g. <code>4</code> , <code>15</code> , <code>-10</code>).
2. User Entered	The non-numeric alphabetical text <code>'d'</code> .
3. Failed Operation	<code>int('d')</code> conversion on statement: <code>l = int(input("l:"))</code> .
4. Why It Failed	The character <code>'d'</code> is an alphabetical string that cannot be parsed as a base-10 numerical digit.
5. Exception Generated	<code>ValueError: invalid literal for int() with base 10: 'd'</code> .
6. How to Fix	Enter whole digits without letters or quotes when prompted, or use <code>str(input())</code> to accept text strings.
7. Corrected Example	Enter a valid integer such as <code>4</code> .

4. Frontend User Interface & Visualizer System

 **Adaptive Theme Support:** Full Light Theme (clean slate/blue) and Dark Theme (deep charcoal #12141a / neon cyan) with instant toggle and persistent browser preferences.

Idle & Execution State Transitions

- **Idle State (Before Run):** The right pane displays a custom vector illustration of an ice-fisherman explorer with the prompt *"Run to view execution logs"*. No execution traces run until the user explicitly clicks **Run**.
- **Running State:** Clicking **Run** activates the line-by-line stepper, interactive compiler output terminal, memory visualizer, and step explanation card. The button toggles to **Stop** (red).
- **Stop / Reset:** Clicking **Stop** halts execution, clears all input buffers, resets variables, and returns cleanly to the idle screen.

5. API Endpoints & Communication Protocols

Endpoint	Method	Payload	Description
/api/trace	POST	{ code, language, customInputs }	Executes code through the native trace engine and returns step-by-step memory states, compiler banner, and diagnostics.
/api/explain	POST	{ code, language }	Performs AST semantic classification, algorithmic pattern matching, and complexity proofs.
/api/debug	POST	{ code, language }	Runs multi-language static analysis and returns syntax issues with side-by-side patch diff fixes.
/api/ocr	POST	multipart/form-data (image)	Extracts and formats code from photos or screenshots with indentation restoration.
/api/share	POST	{ code, language, title, trace }	Creates a persistent shareable session URL.

6. Installation, Local Setup & Docker Deployment

Local Development

```
# 1. Clone the repository
git clone https://github.com/dinesh2505034/codelens-ai.git
cd codelens-ai

# 2. Install Node.js dependencies
npm install

# 3. Build frontend bundle
npm run build

# 4. Start backend server
node server/index.js
# Access application at http://localhost:3001
```

Production Docker Deployment

```
# Build and run Docker container
docker build -t codelens-ai .
docker run -d -p 3001:3001 --name codelens-app codelens-ai
```

7. Security, Sandboxing & Performance Optimization

- **Subprocess Execution Limits:** Subprocess execution in `universalTraceEngine` enforces strict 4000ms timeouts and 10MB buffer limits to prevent infinite recursion and denial-of-service hanging.
- **Client-Side 60fps Performance:** Memory diagrams and code line pointer transitions are GPU-accelerated with CSS hardware transformations and virtualized DOM rendering.
- **100% On-Premise Privacy:** Runs entirely on native backend instances without routing user source code to third-party cloud APIs.